Failure-by-Language: Reproducible Evaluation of Python-like Confusion Language in LLM Code Generation

Jaeyeong CHOI
DGIST EECS
jaeyeong2022@dgist.ac.kr

March 23, 2026

Abstract

Recent studies on AI-assisted coding suggest a gap between short-term productivity and durable comprehension. This paper proposes a reproducible benchmark for syntax-constrained code generation: LLMs are asked to write Python-like code under transformed-token rules.

We present an OCaml/CLI pipeline with lineage-aware schemas, run summary validation, and metric snapshot automation. Our current empirical status is constrained by live API authentication issues, but fixture-based replay and failure-taxonomy analysis provide a controlled baseline for further model comparisons.

1 Introduction

Recent studies on AI-assisted coding consistently show a tension between short-term coding speed and durable understanding, especially for novices.[1]

This work studies a simple but strict question: when a model is instructed to write code in a Python-like language with transformed syntax, does it follow the new language rules, or revert to canonical Python conventions?

We define a reproducible pipeline in three layers:

1. **Specification layer**: one-to-one token aliases and collision constraints.
2. **Execution layer**: OCaml-based transform/validate/roundtrip tooling.
3. **Measurement layer**: summary validation, metric generation, and schema checks.

Our objective is not model ranking. It is to isolate where and how models fail under constrained language perturbation.

2 Related Work

2.1 Code generation foundations

Early code-generation benchmarks and model families (e.g., HumanEval and MBPP) frame our problem context.[Chen et al., 2021, Austin et al., 2021]

[1] See structured review notes in `docs/research/literature/v1-curated.md`.

2.2 Model robustness under constrained syntax

Recent LLM code models (e.g., pretraining methods such as *CodeBERT*, *CodeT5*, *UniXcoder*) have improved language-to-code transfer, but the gap between learned natural-language priors and constrained syntax remains underexplored.[Feng et al., 2020, Wang et al., 2021, Guo et al., 2022]

2.3 Instruction following and alignment

Instruction-tuning and human-feedback approaches increased compliance for broad tasks, yet their effect on syntax-preserving transforms has not been systematically validated under alias perturbation settings.[Ouyang et al., 2022, Brown et al., 2020]

2.4 Repair and edit-style generation

Studies on infilling/editing architectures (e.g., InCoder) suggest that bidirectional context can help fix local code regions, which is relevant to our roundtrip-based mismatch diagnosis pipeline.[Fried et al., 2022]

2.5 Task realism and software-engineering benchmarks

Beyond short-function benchmarks, SWE-bench shows that real-world patching tasks remain far more difficult and expose hidden fragility in supposedly strong models, motivating strict schema and lineage controls for our pipeline.[Jimenez et al., 2023]

2.6 Type/constraint-aware generation

Emerging work on type-constrained decoding indicates a promising direction for enforcing semantic validity during generation, complementing our syntax-focused alias-control strategy.[Mündler, 2025]

3 Method

3.1 Language setup

We use a Python-like confusion language where selected Python tokens are replaced by aliases. We enforce:

- one-to-one mapping between original token and alias,
- no alias sharing across control keywords (e.g., if/elif),
- token-boundary replacement and protection of comments/strings.

3.2 Pipeline

The implementation consists of an OCaml toolchain and Python utility scripts:

- Validation and transformation in `ocaml-confusion-lang`,
- summary/mapping scripts in `ocaml-confusion-lang/scripts`,
- metric aggregation and schema checks via `batch_report_summary.py`, `generate_metric_snapshot.py`, `validate_summary_payload.py`, `validate_metric_schema.py`.

3.3 Metrics

We report:

- ACR (Alias Compliance Rate)
- PRR (Python Reversion Rate)
- ESR (Execution Success Rate)
- MFB (Manual Fix Burden)
- LGP (Learning Gain Proxy)

3.4 Experiment design

Inputs are referenced by task-set IDs and alias IDs. The main fixture path is `fixture-task-set-v1` with `fixture-alias-v1`. Prompt condition uses `strict`. Real model calls are attempted through the batch script path, while fixture replays are used when API access is not available.

3.5 Code-level reliability controls

Recent commits added extensive output provenance states (retention flags/codes, truncation flags, one-to-one alias metadata) and multiple compact formats (JSON/JSONL/TSV/markdown), enabling robust machine parsing and reducing interpretation drift.

4 Experiments

4.1 Run protocol

Each experiment run follows three steps:

1. Prepare task-set and prompt condition metadata.
2. Run model batch or fixture replay path.
3. Validate summary and metric artifacts; archive results under `docs/research/results/`.

4.2 Implemented execution coverage

Our current execution coverage includes:

- fixture summary replay (full artifact generation and schema validation),
- CLI regression coverage expansion on planner output formats and alias aliases,
- structured batch result logging (including `.json`, `.csv`, and `.md`).

4.3 Observed blocking conditions

Batch calls using live LLM endpoints are blocked by authentication failures (HTTP 401), while local OCaml-based roundtrip execution was also blocked when environment dependencies (e.g., dune) were missing at test time.

4.4 Reproducibility protocol

We track run state with committed artifacts and update policy-driven logs. For every change, related tests and schema checks are executed where relevant.

5 Results

Metric	Value	Note
ACR	0.333	fixture replay (task-set: fixture-task-set-v1)
PRR	0.667	same run condition
ESR	0.667	same run condition
MFB	65.0	manual correction burden estimate
LGP	0.667	learning proxy metric from replay context

Table 1: Initial aggregate metrics from replayed fixture run.

5.1 Batch execution attempts

We attempted a 12-prompt real-call batch against gpt-4o-mini from archived prompt versions. The run produced:

- **passed=0, failed=12**
- failure reason: HTTP 401 unauthorized for all requests.

5.2 Failure taxonomy observed

Dominant mismatches in replayed summaries include:

- `line_count_mismatch`
- `whitespace_or_blankline_drift`
- `token_stream_mismatch`
- `token_substitution_mismatch`

This indicates both semantic drift and surface-form instability across model outputs.

6 Discussion

The strongest contribution of this stage is not yet final model performance claims, but an operationally robust benchmark scaffold.

6.1 What we validated

We successfully validated replay-side reproducibility: fixture summaries and metric snapshots are generated, versioned, and schema-checked. Planner CLI coverage for state-code formats expanded significantly, which reduced parser ambiguity and enabled compact, automation-friendly consumption.

6.2 What is currently limited

- All live model runs are currently blocked by API authentication issues (401).
- Some experiments are constrained by missing local build tooling at points of testing.
- Therefore, generalization across model families is not yet established.

6.3 Practical implication

For researchers building language-perturbation benchmarks, provenance and schema discipline are essential. Without this layer, observed failures can be misattributed to models while actually coming from transform, formatting, or execution drift.

7 Conclusion

This project establishes a reproducible confusion-language benchmark stack with explicit collision-safe alias policies, robust output provenance, and schema-enforced measurement artifacts. We also integrated planning and execution logs into a long-run research workflow.

Current quantitative claims are therefore partial: fixture replay indicates non-trivial non-compliance and substantial Python reversion under transformed syntax conditions, but broad model-level claims require renewed API access and larger batch coverage.

Future work focuses on: 1) real batch expansion after valid credentials, 2) model-condition matrix comparisons (base/strict/few-shot/2-step), and 3) educational impact reporting with manual-fix and learning gain tracks.

8 Reproducibility and artifact availability

All scripts, logs, and metrics are versioned in the repository. Representative paths:

- Tooling: `ocaml-confusion-lang/`
- Results: `docs/research/results/`
- Logs: `docs/research/log/`
- Status snapshot: `docs/research/context-state.json`
- Paper draft: `docs/research/papers/v2.md`

References

Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Tom Brown et al. Language models are few-shot learners. arXiv preprint arXiv:2005.14165, 2020. Transformer scaling and in-context learning baseline for subsequent instruction-following work.

Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Zhangyin Feng et al. Codebert: A pre-trained model for programming and natural languages. In *EMNLP-Findings*, 2020.

Daniel Fried et al. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022.

Daya Guo et al. Unified cross-modal pre-training for code representation, 2022.

Carlos Jimenez et al. Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

et al. Mündler. Type-constrained code generation with language models, 2025. Type-aware decoding constraints for code generation (emerging work).

Long Ouyang et al. Training language models to follow instructions with human feedback, 2022.

Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, 2021.